

keyof

```
type Object = {  
  first: 1;  
  second: 2;  
}  
  
type Result = keyof Object; // "first" | "second"
```

Single type

```
type Obj = {  
  0: "a",  
  1: "b",  
  prop0: "c",  
  prop1: "d"  
}  
  
type Result0 = Obj["prop0"] // "c"  
type Result1 = Obj[0 | 1] // "a" | "b"  
type Result2 = Obj["prop0" | "prop1"] // "c" | "c"
```

Getting values from object

```
type Obj = {  
  a: "A";  
  b: "B";  
  c: number;  
}  
  
type Values = Obj[keyof Obj] // number | "A" | "B"
```

Unions

```
type A = "a" | "b" | "c"  
type B = "b" | "c" | "d"  
  
type Union = A | B // "a" | "b" | "c" | "d"
```

intersections

```
type A = "a" | "b" | "c"  
type B = "b" | "c" | "d"  
  
type Union = A & B // "b" | "c"
```

Union with Objects.

```
type ObjectA = {  
  first: number;  
  sharedProp: string;  
}  
type ObjectB = {  
  second: boolean;  
  sharedProp: string;  
}  
  
type Union = ObjectA | ObjectB // only sharedProp
```

Conditionals

```
type Wrap<T> = T extends {length: number} ? [T] : T  
  
type IsAssignableTo<A, B> = A extends B ? true : false;  
  
// Type 123 is assignable to type "number"  
type Result1 = IsAssignableTo<123, number> // true  
  
// Type "number" is not assignable to type 123  
type Result2 = IsAssignableTo<number, 123> // false
```

exclude

```
type Exclude<T, U> = T extends U ? never : T  
  
type Res0 = Exclude<1 | 2 | 3, 2>; // 1 | 3  
  
type Res1 = Exclude<"a" | 2 | "b", number>; // "a" | "b"  
  
type Res2 = Exclude<1 | "a" | 2 | "b", 1 | "b" | "c">; // "a" | 2
```

Extract

```
type Extract<T, U> = T extends U ? T : never;  
  
type Res0 = Extract<1 | "a" | 2 | "b", number> // 1 | 2  
  
type Res1 = Extract<1 | "a" | 2 | "b", 1 | "b" | "c"> // 1 | "b"
```

String manipulation

```
type Res0 = Uppercase<"test">  
type Res1 = Lowercase<"TEST">  
type Res2 = Capitalize<"test">  
type Res3 = Uncapitalize<"Test">
```

Pick

```
type Obj = {  
  john: 1;  
  paul: 2;  
  george: 3;  
  ringo: 4;  
};  
  
// {john: 1, paul: 2}  
type Res = Omit<Obj, "george" | "ringo">;  
  
// {george: 3, ringo: 4}  
type Res1 = Pick<Obj, "george" | "ringo">;
```

ReactComponent type

```
type ComponentProps = React.ComponentProps<typeof MyComponent>
```

Record

```
const serviceConfig: Record<string, string | number | boolean> = {  
  port: 3000,  
  basePath: "http://localhost",  
  enableStripePayments: false,  
  somethingelse: true // OK  
};  
  
console.log(serviceConfig.port); // prints 3000  
  
type ServiceConfigParams = "port" | "basePath" | "enableStripePayments";  
  
const serviceConfigChecked: Record<ServiceConfigParams, string | number | boolean> = {  
  port: 3000,  
  basePath: "http://localhost",  
  enableStripePayments: false,  
  somethingelse: true // ERROR  
};
```

Partial

Partial - передавати чи ні решта параметрів

```
interface TodoItemProps {
  title: string;
  description: string;
  priority: string;
}

function partial(todo: Partial<TodoItemProps> = {}) {
  const ob = Object.assign({}, todo);
  console.log(ob);
}

partial({ title: "Hello" }); // {title: 'Hello'}

// { title: "Hello", description: "Description" }
partial({ title: "Hello", description: "Description" });
```

Required

протилежність Partial

```
interface TodoItemProps {
  title: string;
  description: string;
  priority: string;
}

function partial(todo: Required<TodoItemProps>) {
  const ob = Object.assign({}, todo);
  console.log(ob);
}

partial({ title: "Hello" }); // ERROR
partial({ title: "Hello", description: "Description", priority: "low" });
```